Project Risk, Quality, and Communication Plan

Project name: Smart parking platform

Course: CIS 4374 – Software Project Management

Date: 10/15/26

1. Project Risk Identification

1.1 Technical Risks

- **Risk T-01 (IoT Sensor Telemetry Loss & Stale State):** Physical ultrasonic/optical stall sensors or gate barrier controllers may drop off the wireless network or transmit delayed packets, creating phantom parking availability and misrepresenting live capacity.
- **Risk T-02 (High-Concurrency Reservation Race Conditions):** Multiple simultaneous driver checkout requests for the final remaining stall in a popular garage could bypass naive validation and cause duplicate confirmed bookings.
- **Risk T-03 (Payment Processing Gateway Latency & Webhook Failures):** Gateway timeouts or dropped webhook callbacks during high-volume event hours could charge users without issuing valid digital access passes / QR codes.
- **Risk T-04 (Third-Party Geolocation & Mapping API Rate Overages):** Exceeding map service query rate limits during peak commuting windows degrades in-app turn-by-turn routing and nearby garage discovery.

1.2 Schedule Risks

- **Risk S-01 (Gate Controller Hardware / API Integration Delays):** Vendor delays in providing sandbox APIs, hardware specifications, or authentication keys for parking barrier equipment stall Sprint delivery.

- **Risk S-02 (Scope Creep on Dynamic Pricing / Analytics Logic):** Over-engineering complex surge pricing or predictive analytics early in development delays core driver reservation workflows.
- **Risk S-03 (Hardware Test Harness Mocking Overhead):** Building custom mock environments to simulate barrier operations and License Plate Recognition (LPR) cameras takes longer than planned, delaying end-to-end integration testing.
- **Risk S-04 (Mobile App Store Compliance & Review Bottlenecks):** Rejection cycles from Apple or Google Play regarding precise background location tracking or payment compliance postpone the release milestone.

1.3 Financial Risks

- **Risk F-01 (Cloud Database & Ingestion Cost Overruns):** Real-time sensor polling at high frequencies generates excessive write operations, driving unexpected monthly cloud infrastructure bills.
- **Risk F-02 (Third-Party Map & Messaging API Usage Surges):** Elevated transaction volumes and frequent SMS OTP verification requests scale commercial API expenses beyond early-stage operating budgets.
- **Risk F-03 (Merchant Chargebacks & Dispute Penalties):** Drivers requesting refunds for barrier failures or missed reservations generate dispute handling fees and processing losses.
- **Risk F-04 (Hardware Edge Node Replacement & Sourcing Expenses):** Physical hardware components, microcontrollers, and edge gateways incur unexpected supply chain price inflation or replacement costs during field testing.

1.4 People & Team Risks

- **Risk P-01 (Single-Point Dependency on Cloud & DevOps Architecture):** Critical pipeline setups, deployment environments, and database migrations depend primarily on one engineer, causing work stoppages if that member is unavailable.
- **Risk P-02 (Disparity Between Driver Experience and Operator Tooling):** Development heavily favors end-user mobile interfaces while neglecting essential admin controls required by facility managers.

- **Risk P-03 (Academic Milestones & Exam Capacity Fluctuations):** Midterms, finals, and concurrent project deadlines across team members lead to temporary dips in sprint velocity.
- **Risk P-04 (QA Testing & Test Automation Gaps):** Inconsistent end-to-end regression testing allows breaking API changes to deploy unnoticed to staging builds.

2. Project Risk Registar

Risk ID	Category	Risk Description	Prob (1-5)	Impact (1-5)	Score (Pxl)	Mitigation / Response Strategy	Owner
RSK-01	Technical	High-concurrency race condition causing double bookings on the last available stall.	3	5	15	Implement distributed locking (e.g., Redis mutex) with a 5-minute atomic reservation holding state.	Backend Lead
RSK-02	Technical	Stale parking stall occupancy data due to IoT hardware or network dropouts.	4	4	16	Implement client timestamp indicators, sensor heartbeat health checks, and fallback dead-reckoning logic.	IoT Lead
RSK-03	Technical	Payment processor timeout during peak entry windows.	2	5	10	Use idempotent API requests, asynchronous transaction verification, and instant auto-retry worker queues.	Backend Lead
RSK-04	Schedule	Garage barrier hardware SDK integration takes longer than scheduled sprint allocations.	4	3	12	Construct a mock API service simulating gate raise/lower trigger early in Sprint 1 to keep frontend development unblocked.	Full-Stack Dev
RSK-05	Schedule	Scope creep on advanced dynamic pricing rules delays core checkout launch.	3	4	12	Limit initial release to fixed/hourly rates; defer algorithmic surge pricing to product backlog.	Project Manager

RSK-06	Financial	Cloud database cost spikes caused by continuous real-time telemetry polling.	3	3	9	Implement edge aggregation so controllers transmit messages only when state changes, rather than continuously polling.	DevOps Lead
RSK-07	Financial	High merchant chargeback fees caused by gate barrier read errors.	2	4	8	Provide automated 15-minute grace-period cancellation workflows directly inside the mobile app.	Product Owner
RSK-08	People	Key-person dependency on cloud infrastructure and deployment scripts.	3	4	12	Document configuration as Infrastructure as Code (IaC) and conduct peer cross-training walkthroughs.	DevOps Lead
RSK-09	People	Sprint velocity drops during university exam schedules.	4	3	12	Reduce sprint commitment capacity to 70% during exam weeks and track burnup charts daily.	Project Manager
RSK-10	Schedule	Mobile app stores reject release build due to background location access requests.	3	4	12	Restrict permissions to "while using app" and prepare Apple/Google privacy justification documentation upfront.	Mobile Lead

3. Team communication

3.1 Meeting Cadence and Ceremonies

Meeting / Ceremony	Frequency & Time	Participants	Objective & Deliverable
Daily Standup	Mon-Thu (10-15 mins, async via Slack/Discord or 9:00 AM)	Entire Project Team	Identify progress and blockers: (1) Finished yesterday, (2) Plan for today, (3) Current roadblocks.
Sprint Planning	Bi-weekly (First Monday of sprint, 60 mins)	Full Team & Project Manager	Scope definition: Review Product Backlog, assign story points, commit cards to Sprint Backlog.
Architecture & Technical Sync	Weekly (Wednesdays, 45 mins)	Backend, Frontend, DevOps Leads	Resolve technical dependencies: Review API schemas, database schemas, and integration points.
Sprint Review & Demo	Bi-weekly (Final Friday of sprint, 45 mins)	Team, Course Instructor / Stakeholders	Demonstration: Walk through functioning software features that meet acceptance criteria.
Sprint Retrospective	Bi-weekly (Immediately following Demo, 30 mins)	Entire Project Team	Process improvement: Discuss what worked well, what failed, identify two actionable process updates.

3.2 Reporting Methods & Information Radiators

Work & Task Tracking: Trello / Jira functions as the single source of truth for all user stories, sprint backlogs, bugs, and acceptance criteria.

Source Control & Code Reviews: GitHub manages all version control. Work is committed via branch pull requests requiring at least one peer code review approval prior to merging.

Daily Operational Communication: Discord / Slack structured into distinct channels:

- #announcements – Deadlines, schedule adjustments, and milestones.
- #dev-backend & #dev-frontend – Technical implementation questions.
- #git-activity – Automated repository webhook alerts for commits and PRs.
- #blockers – Immediate operational or academic impediments.

Stakeholder Progress Reporting: Bi-weekly **Executive Status Reports** exported to PDF and shared via the project portal, summarizing:

1. Sprint goals achieved vs. carryover.
2. Sprint burndown and team velocity.
3. High-priority risk register updates (RSK-01 through RSK-03).
4. Goals committed for the upcoming sprint.